

Analysing Requirements Communication Using Use Case Specification and User stories

Ana Carolina Oran, Elizamary Nascimento
Universidade Federal do Amazonas
(UFAM) Manaus – AM – Brazil
{ana.oran, elizamary.souza}
@icompu.fam.edu.br

Gleison Santos
Universidade Federal do Estado do Rio
de Janeiro (UNIRIO)
Rio de Janeiro – RJ – Brazil
gleison.santos@uniriotec.br

Tayana Conte
Universidade Federal do Amazonas
(UFAM) Manaus – AM – Brazil
tayana@icompu.fam.edu.br

ABSTRACT

Effective requirements communication is essential in software development projects due to the importance of understanding the requirements throughout the software development cycle. Software requirements can be specified in different formats, for instance using free texts or more structured forms, such as use cases and user stories used in Behavior Driven Development (BDD). We present a comparative analysis on the requirements communication dynamics using use case specification and user stories as the basis for mockups creation. We carried out an exploratory empirical study involving 16 students. The study comprised 3 steps: requirements specification, mockups construction, and inspection to investigate whether the mockups were in accordance with the specifications. Results show that there is no significant difference in using use case specification or user stories to communicate software requirements. Our findings suggest that different specification formats can provide similar results while communicating requirements, nonetheless the human factor should not be neglected.

CCS CONCEPTS

• **Software notations and tools** → **System description languages**; Unified Modeling Language (UML); Specification languages.

KEYWORDS

Requirements communication, requirements specification, use case, Behavior Driven Development, experimental study.

ACM Reference format:

A. C. Oran, E. Nascimento, G. Santos, T. Conte. 2017. Analysing Requirements Communication Using Use Case Specification and User stories. In *Proceedings of 31st Brazilian Symposium on Software Engineering, Fortaleza, Ceará, Brazil, September 2017 (XXXI SBES)*, 10 pages.
<https://doi.org/10.1145/3131151.3131166>

Publication rights licensed to ACM. ACM acknowledges that this contribution was authored or co-authored by an employee, contractor or affiliate of a national government. As such, the Government retains a nonexclusive, royalty-free right to publish or reproduce this article, or to allow others to do so, for Government purposes only.

SBES'17, September 20–22, 2017, Fortaleza, CE, Brazil

© 2017 Copyright is held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-5326-7/17/09...\$15.00

<https://doi.org/10.1145/3131151.3131166>

1 INTRODUÇÃO

A comunicação dos requisitos é essencial em todos os projetos de software, uma vez que existe necessidade de entendimento das informações durante todo o ciclo do processo de desenvolvimento de software [29]. A comunicação de requisitos é iniciada com a elicitação de requisitos com o cliente e continua ao longo de todo o projeto de desenvolvimento, envolvendo diferentes papéis [33]. Na fase de elicitação de requisitos, os clientes devem ser capazes de comunicar as suas necessidades para os analistas de requisitos e os analistas precisam ser capazes de transmitir essas necessidades para todos os demais membros da equipe de desenvolvimento de forma clara e eficaz. Al-Rawas e Easterbrook [1], Tu *et al.* [33] e Fernández *et al.* [11] afirmam que tanto usuários finais quanto profissionais de software têm dificuldades relacionadas à validação e compreensão das informações resultantes da especificação dos requisitos. Uma boa comunicação não significa apenas emitir os requisitos para todo o grupo, mas também é preciso garantir o entendimento destes na sua recepção. Segundo Hoisl *et al.* [13], a forma como os requisitos são descritos pode influenciar no esforço requerido para que o receptor desta informação entenda o que deve ser desenvolvido. É importante representar os requisitos de forma que permita que todos os stakeholders envolvidos no projeto estabeleçam um entendimento comum sobre o funcionamento do sistema para que o produto final desenvolvido pela equipe esteja de acordo com as expectativas do cliente.

Há diferentes formas de se representar os requisitos de um sistema, desde uso de textos livres a formas mais estruturadas. Problemas com comunicação de requisitos podem surgir devido ao modelo de especificação escolhida para o processo de desenvolvimento do Sistema [15]. Uma forma muito difundida de se descrever requisitos são os casos de uso, bastante utilizados pela indústria [3, 9, 22]. Casos de uso, além de descreverem os requisitos, são utilizados também como meio de comunicação entre os membros da equipe de projeto e outros stakeholders [8] e também como insumos em diferentes atividades não diretamente relacionadas a um processo de Engenharia de Requisitos, como atividades de planejamento, análise, projeto, desenvolvimento, teste e manutenção do sistema [2]. Outra forma que tem despertado interesse nas comunidades acadêmicas e industriais é a *user story* usada em *Behavior Driven Development* (BDD ou Desenvolvimento Dirigido a Comportamento) [8]. BDD é uma metodologia ágil preocupada em utilizar um vocabulário comum para entendimento de

requisitos por todos os stakeholders do projeto [7]. Segundo Silva [27], BDD impulsiona as equipes de desenvolvimento para uma especificação de requisitos baseada em *user stories* [19] em um formato abrangente de linguagem natural. As *user stories* consistem em descrições breves, na perspectiva do usuário final, das funcionalidades desejadas [35] e englobam vários aspectos da especificação de requisitos representados em linguagem natural, tais como: cenários alternativos, de exceção e interação do ator com o sistema.

Com o intuito de comparar a dinâmica da comunicação de requisitos utilizando especificação de caso de uso e *user story* como base para a criação de mockups (representações dos aspectos da interface do usuário [18]), este artigo apresenta os resultados de um estudo experimental exploratório onde foram verificadas a emissão e recepção dos requisitos utilizando caso de uso e *user story*, avaliando e comparando o grau de correção das especificações e dos mockups criados. Para direcionar este estudo, foi definida a seguinte questão de pesquisa: “O uso de casos de uso ou *user stories* afeta a comunicação dos requisitos para a construção de mockups de forma diferente?”.

O restante deste artigo está organizado da seguinte forma: a Seção 2 aborda conceitos sobre processo de engenharia de requisitos, caso de uso, *user story* utilizada no BDD e trabalhos relacionados; a Seção 3 descreve o planejamento e execução do estudo experimental; a Seção 4 apresenta a análise dos resultados encontrados neste estudo; a Seção 5 as discussões e ameaças à validade do estudo. Por fim, a Seção 6 apresenta as conclusões, limitações e trabalhos futuros.

2 ESPECIFICAÇÃO DE REQUISITOS DE SOFTWARE

O documento de especificação de requisitos é uma declaração dos requisitos para os stakeholders. Este documento fornece base para a análise e validação de requisitos junto aos stakeholders, definição do que os projetistas têm que construir, bem como a verificação da qualidade do produto final no momento da entrega [5,19]. Através de um mapeamento sistemático da literatura sobre comunicação de requisitos de software, identificou-se que existem diferentes formas para representar os requisitos de software e a escolha por uma dessas formas dependerá de fatores como, a experiência da equipe e a visibilidade destes requisitos [31]. Entre muitas formas de representar requisitos, duas das mais utilizadas pela indústria são: casos de uso [3, 17] e *user stories*, devido ao crescimento do desenvolvimento ágil [34]. A seguir é apresentada uma breve descrição destas formas de representar os requisitos.

2.1 Caso de uso

Um dos artefatos utilizados pelos engenheiros de software para descrever e documentar requisitos do software é a especificação de Caso de Uso (UC) [31]. Originalmente proposto por Jacobson *et al.* [16] como uma forma dos profissionais de software obterem uma melhor compreensão dos requisitos de um sistema. Segundo Bezerra [6] e Cockburn [9], caso de uso é a especificação da sequência de interações entre um sistema e um

ou mais atores (agentes externos a esse sistema) de forma sequencial, representando um relato de utilização de alguma funcionalidade do sistema em questão, sem revelar a estrutura e o comportamento internos desse sistema.

A estrutura amplamente utilizada pelos engenheiros de requisitos em geral utiliza a seguinte estrutura [24]: *Nome*: descreve o nome do caso de uso, capturando a sua essência. *Descrição*: uma descrição sucinta do caso de uso, descrevendo o objetivo do caso de uso. *Pré-condições (opcional)*: descrição de condições que devem ser atendidas antes de iniciar o caso de uso. *Fluxo Principal*: descreve os passos do caso de uso realizados em situações normais, ou seja, sem considerar nenhum erro. *Fluxos Alternativos*: descrevem formas alternativas de realizar certos passos do caso de uso. *Fluxos de Exceção*: descrevem formas de recuperação de erros que podem ocorrer em certos passos do caso de uso. *Pós-condições (opcional)*: descrição de condições que devem ser atendidas após a execução do caso de uso, considerando que o fluxo de eventos normal seja realizado com sucesso. *Regras de Negócio*: são políticas, procedimentos ou restrições que devem ser levadas em consideração durante a execução do caso de uso.

Esta estrutura é voltada principalmente para descrever os fluxos principal e alternativo de eventos. O uso dessa estrutura facilita o entendimento da funcionalidade por utilizar uma estrutura pré-definida [4] e permite aos engenheiros de software flexibilidade na especificação dos requisitos [31] ajudando assim na comunicação de requisitos dentro das equipes de desenvolvimento.

2.2 *User story* usada em Behavior Driven Development (BDD)

Behavior-Driven Development é uma metodologia ágil que auxilia grupos a construir e entregar software de maior valor e qualidade de forma mais rápida [28]. BDD tem como objetivo promover o entendimento comum do domínio do negócio entre os stakeholders do projeto, além de apoiar a automação de outras atividades do ciclo de desenvolvimento e manter a documentação atualizada [10, 12, 28].

A especificação dos requisitos em BDD é feita por todos os stakeholders do projeto no formato de *user stories* (US). Onde cada *user story* é instanciada com múltiplos cenários BDD, que representam um comportamento esperado para uma determinada situação. Os cenários BDD possuem um template, escrito em uma linguagem independente de domínio, de forma a ser utilizado em qualquer situação [7]. Segundo Silva [27], as *user stories* utilizadas em BDD são desenvolvidas sob uma perspectiva de comportamento no ponto de vista do usuário. Este método promove uma descrição de linguagem natural semiestruturada, de forma não ambígua, além de promover a reutilização de comportamentos de negócios que podem ser compartilhados para vários recursos no sistema.

Um cenário BDD é composto por uma parte inicial que representa uma *user story* [7]: *Como*: descreve o papel/ator que utilizará a funcionalidade. *Eu quero*: descreve o objetivo, desejo ou característica da funcionalidade. *Para que*: descreve o motivo, justificativa ou benefício da funcionalidade.

A segunda parte do cenário descreve o comportamento em si e utiliza-se palavras chaves: *Dado que*: descreve um contexto inicial, descrito por várias características, são as pré-condições para executar o cenário. *Quando*: descreve o que pode eventualmente, ocorrer uma mudança no domínio, os testes (passos) para execução do cenário. *Então*: descreve o estado final pretendido, o resultado esperado da execução dos passos.

Esta estrutura utiliza uma linguagem ubíqua que permite especificar o comportamento do sistema ao invés de detalhes técnicos, o que possibilita a comunicação, colaboração e compreensão dos requisitos do sistema por todos os envolvidos no desenvolvimento do software [7, 8, 10, 12, 27, 28].

2.3 Trabalhos relacionados

Lauesen e Kuhail [17] compararam a técnica de descrição de tarefas com casos de uso em projetos reais e verificaram que a utilização de casos de uso torna o requisito muito restritivo por duas razões: 1. Força o analista a criar um diálogo muito cedo, projetando assim uma solução em vez de especificar uma necessidade do cliente. 2. O template utilizado para especificação pode induzir o analista a criar regras, pré-condições e fluxos que muitas vezes não refletem a necessidade do cliente e pode prejudicar no desenvolvimento do sistema. Por outro lado, na descrição de tarefas não existe o template para ser seguido pelo analista, a descrição é criada ao longo do desenvolvimento e descreve-se apenas a interação do usuário com o sistema.

Tu *et al.* [33] afirmam que o uso de documentos mais transparentes, ou seja, com uma maior visibilidade de informações para as partes interessadas, possa contribuir para uma comunicação mais eficaz. Eles realizaram um experimento que mostrou evidências de que uma maior transparência dos documentos de requisitos leva a uma comunicação mais eficaz.

Al-Rawas e Easterbrook [1] descrevem um estudo sobre os problemas de comunicação de requisitos em projetos de desenvolvimento de software. Os resultados mostraram que as questões organizacionais e sociais têm grande influência na eficácia das atividades de comunicação. Além disso, apontam que os usuários finais têm dificuldades em entender e validar os requisitos devido às notações utilizadas. Por outro lado, os profissionais de software relatam que o uso de notações que são legíveis para os seus clientes, muitas vezes, gera documentos extensos e que apresentam ambiguidade.

Os trabalhos apresentados focaram na comunicação de requisitos entre cliente e analista. Estes não tiveram como objetivo verificar qual a melhor forma de representar os requisitos para que todos da equipe tenham o entendimento do que será desenvolvido.

3 ESTUDO EXPERIMENTAL

Com o propósito de comparar especificações de casos de uso e *user stories* foi realizado um estudo experimental para comparar as abordagens em termos de comunicação de requisitos entre os membros da equipe de desenvolvimento. Esta seção descreve o plano de estudo e sua execução. A versão completa do pacote experimental encontra-se disponível no relatório técnico [23].

3.1 Planejamento do Estudo

Para orientar o projeto do estudo, a partir da questão de pesquisa (O uso de casos de uso ou *user stories* afeta a comunicação dos requisitos para a construção de mockups de forma diferente?), foi definido o objetivo da seguinte forma:

“Analisar as especificações de caso de uso e *user story* usadas em BDD com a finalidade de compará-las em relação a comunicação de requisitos dentro de grupos de desenvolvimento, do ponto de vista de pesquisadores em engenharia de software no contexto de especificação de requisitos e construção de mockups por estudantes de graduação e pós-graduação”.

3.1.1 Participantes. O estudo foi realizado em um ambiente acadêmico, com 16 participantes, sendo 11 alunos de graduação e 5 alunos de pós-graduação do curso de Ciência da Computação e Sistema de Informação da Universidade Federal do Amazonas (UFAM). Os alunos haviam cursado as disciplinas de Engenharia de Software e Análise e Projeto de Sistemas e estavam cursando a disciplina de Engenharia de Software Experimental.

Os alunos foram caracterizados como novatos, pois tinham apenas experiência acadêmica com a especificação de caso de uso e *user story*. Somente um participante da pós-graduação já havia trabalhado *user story* e com caso de uso na indústria.

3.1.2 Artefatos. Todos os participantes assinaram um formulário de consentimento - Termo de Consentimento Livre e Esclarecido (TCLE), em que concordaram em fornecer seus resultados para análise. Os artefatos utilizados neste estudo experimental foram: a) descrição textual de quatro cenários de aplicações com níveis de complexidade semelhantes; b) modelo de especificação de caso de uso e *user story*; c) artefatos para criação da especificação; d) artefatos para criação dos mockups; e) artefatos para a inspeção dos mockups; f) artefato para descrição de e-mail de comunicação; g) questionários de avaliação de especificação, avaliação de mockup e avaliação geral. Os 4 cenários utilizados no estudo foram:

Cenário 1: Um sistema de restaurante para pessoas com restrição alimentar onde é possível clientes procurarem restaurantes que oferecem comidas diferenciadas e também realizar o pedido sem sair de casa. As funcionalidades do sistema são: cadastro de cliente, pesquisar restaurantes especializados em algum tipo de restrição alimentar e realização de pedido online.

Cenário 2: Um sistema de planejamento de viagens para as pessoas que criam seus roteiros de viagens. As funcionalidades do sistema são: cadastro de roteiros de viagens, cadastro de informações de voos, reserva de hotéis e controle de dinheiro.

Cenário 3: Um sistema de acompanhamento de rotinas de idosos para as famílias e para os idosos terem seus dados de rotina disponível para consulta. As funcionalidades do sistema são: cadastro de consultas médicas, cadastro e lembrete de remédios utilizados e registro de informações do idoso.

Cenário 4: Um sistema para facilitar a troca de livros entre seus usuários. As funcionalidades do sistema são: cadastro de livros, disponibilizar livros para troca e gerenciar de troca.

3.1.3 Treinamento. Todos os participantes receberam treinamento de 2h30min em um mesmo ambiente sobre os dois tipos de especificação. Durante o treinamento também foram realizados exercícios práticos de especificação em casos de uso e

user stories.

3.1.4 *Inspeção dos mockups*. Para identificar os defeitos, foi criado um formulário de inspeção baseado em Travassos *et al.* [32] e no Método de Avaliação de Comunicabilidade (MAC) [25], conforme apresentado na Tabela 1. Foram adotadas essas nomenclaturas diferenciadas do MAC para despertar o interesse dos alunos na atividade de inspeção dos mockups.

Tabela 1: Classificação de defeitos entre especificação e Mockups

Categoria	Descrição	Exemplo
Onde tá?	Informações que foram descritas na especificação e não foram inseridas no mockup	Na especificação tem a descrição: “O cliente seleciona opção Salvar”. No mockup não tem o botão Salvar
Não era assim!	Informações que foram descritas na especificação e inseridas no mockup de outra forma	Na especificação tem a descrição: “O cliente seleciona a opção Salvar”. No mockup o nome do botão é Inserir
O que é isso?	Informações que foram inseridas no mockup e que não estavam descritas na especificação	No mockup tem a opção Salvar e Cancelar. Na especificação só tem a descrição da opção Salvar.
Falta de dependência!	Informações que foram descritas como um cálculo ou informação derivada de outra que no mockup não obedeceram essa dependência	No mockup tem que mostrar um campo idade e preço da entrada do cinema, ambos dependentes da data de nascimento. Ou seja, campo idade e preço deve ser calculado a partir da data de nascimento e não inseridos.

3.1.5 *Inspeção da especificação*. Para avaliar as especificações de casos de uso e *user stories* geradas pelos grupos, foi criado um checklist de inspeção. A técnica de checklist foi escolhida por ser uma das mais utilizadas para inspecionar casos de uso [4] e foi adaptada para inspecionar também as *user stories*. Com a inspeção é possível reduzir esforços em relação ao retrabalho na correção de defeitos em artefatos durante o desenvolvimento de software [20]. Desse modo, foi elaborado um checklist de inspeção baseado em Travassos *et al.* [32] e Anda *et al.* [3] com o objetivo de identificar defeitos na especificação de casos de uso e *user stories*, como, omissão de regras de negócio, omissão de fluxos alternativos e fluxo de exceções para caso de uso e omissão de cenários para *user stories*. Travassos *et al.* [32] apresentam as classes de defeitos que comumente são encontrados em artefatos de software: Omissão (Informações necessárias foram omitidas do caso de uso), Fato Incorreto (Algumas informações no caso de uso contradizem a lista de requisitos ou do conhecimento geral do domínio do sistema), Inconsistência (As informações em uma parte do caso de uso estão inconsistentes com outras no caso de uso), Ambiguidade

(As informações no caso de uso são ambíguas, isto é, é possível ao cliente, desenvolvedor ou testador interpretar as informações de diferentes maneiras podendo não levar a uma implementação correta) e Informação Estranha (As informações fornecidas não são necessárias para o caso de uso). A técnica de Anda *et al.* [3] apresenta itens de verificação que auxiliam de forma mais detalhada quais informações as especificações de caso de uso devem conter para atender aos requisitos de qualidade de um caso de uso, esses itens foram adaptados para a *user story*. Desta forma os itens de verificação foram organizados seguindo a taxonomia de Travassos *et al.* [32] como mostra a Tabela 2, itens de verificação para especificação de caso de uso e itens de verificação para especificação de *user story*. Os checklists foram revisados por três pesquisadores antes de serem utilizados para a inspeção das especificações.

Tabela 2: Exemplos dos Itens de Verificação para Caso de Uso (UC) e User story (US)

Categoria	Código	Itens de Verificação
Omissão	UC_OMI_01	Faltou identificar/descrever o nome do(s) ator(es)?
	US_OMI_01	Faltou descrever a <i>user story</i> do cenário?
Fato incorreto	UC_FIN_01	O nome dado ao caso de uso expressa corretamente seu objetivo?
	US_FIN_05	A descrição no campo “Dado que” do cenário BDD está incompleta/incorreta?
Inconsistência	UC_INC_03	O sequenciamento nos fluxos principal, alternativos e de exceções estão coerentes?
	US_INC_02	A descrição do cenário está inconsistente com o comportamento da funcionalidade do sistema?
Ambiguidade	UC_AMB_01	Os nomes de atores refletem seus papéis? Ou estão ambíguos (podem levar a dupla interpretação)?
	US_AMB_01	O nome do cenário permite interpretações diferentes do seu objetivo?
Informação estranha	UC_IES_02	Informações descritas nas regras de negócio fazem parte do contexto do caso
	US_IES_01	Informações descritas nos cenários não fazem parte do contexto das funcionalidades do sistema?

Foram criados graus de severidade para cada tipo de defeito. Por exemplo, a severidade do tipo “Grave” foi utilizada para classificar defeitos de omissão, ou seja, informações que não foram descritas no caso de uso e na *user story*. Os defeitos de severidade “Média” foram utilizados para classificar informações que não foram descritos por completo ou que foram descritos de

forma incorreta. Os defeitos de severidade “Baixa” identificaram aqueles defeitos que não prejudicavam a compreensão e entendimento do caso de uso e da *user story*.

3.2 Execução do Estudo

Este experimento foi executado em 3 etapas: 1) especificação dos requisitos, 2) construção dos mockups e 3) inspeção para investigar se os mockups estavam de acordo com o que foi especificado. Na etapa 1 os participantes foram divididos aleatoriamente em quatro grupos (1 a 4) com 4 pessoas cada e receberam os cenários para fazerem as especificações com *user story* e caso de uso e modelos para serem seguidos. Neste ponto, destaca-se que apesar de cada cenário conter várias funcionalidades, cada grupo especificou apenas duas com grau de complexidade equivalentes entre si. A [Tabela 3](#) apresenta a divisão dos grupos, cenários e especificações durante o estudo.

Tabela 3: Distribuição dos grupos, cenários e especificações

Grupos	Cenários	Tipos de especificação
1	1	<i>User story</i>
2	2	<i>User story</i>
3	3	Caso de uso
4	4	Caso de uso

Na etapa 2 a principal atividade foi a construção dos mockups baseados nas especificações criadas. Destaca-se que cada grupo recebeu uma especificação de caso de uso e *user story* diferente da que especificou na etapa 1, por exemplo, equipe 1 recebeu um cenário e fez a especificação de requisitos na etapa 1, na etapa 2 de construção de mockups, a equipe 1 recebeu as especificações da equipe 2 e 3 para construir o mockup, conforme mostrado na [Tabela 4](#). Durante a construção dos mockups, os grupos podiam tirar dúvidas com o grupo especificador através de bilhetes. Para isto, foi utilizado uma folha de papel onde o grupo de construção escrevia sua dúvida, o moderador entregava-o para o grupo especificador responder o questionamento e, por fim, o papel era devolvido para a grupo de construção. Esse procedimento foi inserido no estudo com o intuito de evitar a propagação de defeitos inseridos na especificação para os mockups.

Tabela 4: Distribuição de construção dos mockups

Grupos	Construção de Mockups dos cenários
1	3 e 2
2	4 e 1
3	1 e 4
4	2 e 3

Nesta etapa os grupos não fizeram o mockup das especificações que tinham criado no anterior, havendo o cuidado de garantir o rodízio da especificação. Por exemplo, quem especificou com *user story* construiu o primeiro mockup utilizando uma especificação de caso de uso e depois construiu um mockup com *user story*. Dessa forma, procurou-se reduzir o

viés de aprendizado no tipo de especificação.

Na etapa 3 foi realizada a inspeção dos mockups pelos grupos que fizeram a especificação. Os grupos que criaram as especificações receberam os mockups criados pelos outros grupos para identificar os defeitos entre o que foi especificado por eles e o que foi criado nos mockups pelos grupos de construção. Dessa forma, o grupo 1 verificou os defeitos dos mockups criados pelos grupos 3 e 2. O grupo 2 verificou os defeitos dos mockups criados pelos grupos 4 e 1. O grupo 3 verificou os defeitos dos mockups criados pelos grupos 1 e 4. E o grupo 4 verificou os defeitos dos mockups dos grupos 2 e 3.

4 ANÁLISE DOS RESULTADOS

4.1 Resultados Quantitativos

Nesta seção, são apresentados os resultados quantitativos referentes à análise, feita pelos pesquisadores, das especificações de caso de uso e *user stories* criada pelos participantes, análise dos defeitos cometidos ao criar os mockups a partir das especificações e a relação entre as especificações e mockups criados, visando à comunicação dos requisitos dentro dos grupos.

4.1.1 Análise da especificação. Para avaliar a correteza das especificações, criadas pelos participantes do estudo, os pesquisadores realizaram inspeções nas especificações usando os checklists de inspeção para auxiliar na verificação de defeitos da especificação de casos de uso e *user stories* de forma guiada. Cada item identificado como incorreto, impactou no grau de correteza da mesma. O processo de avaliação das especificações foi realizado em duas etapas. Na primeira etapa dois, pesquisadores avaliaram as especificações que os participantes elaboraram a partir dos cenários entregues (duas especificações de casos de uso e duas especificações *user stories*). Na segunda etapa, três pesquisadores revisaram por completo a avaliação realizada e retiraram as discrepâncias. A [Tabela 5](#) apresenta uma parte dessa avaliação.

A avaliação das especificações de caso de uso e *user story* resultou em 45 defeitos, dos quais, 24 defeitos foram identificados nas especificações de *user stories* dos grupos 1 e 2 e 21 defeitos foram identificados nas especificações de casos de uso dos grupos 3 e 4. A [Fig. 1](#) mostra o resultado quantitativo dos diferentes tipos de defeitos identificados nas especificações.

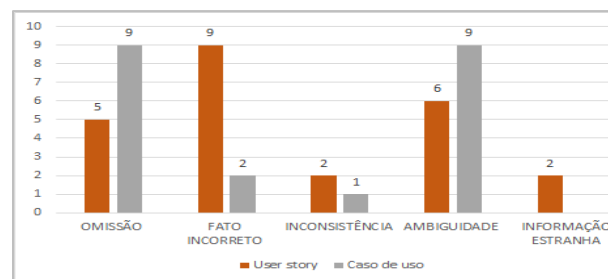


Figura 1: Números de defeitos encontrados nas especificações de user stories e caso de uso.

Tabela 5: Classificação da avaliação das especificações

Cód. do defeito	Descrição do defeito encontrado	Nome do UC	Elem. do template
UC_AMB_04	Na pré-condição ficou estranho à frase: “Estar logado” – O usuário deve estar logado, como descrito no caso de uso Cadastrar um livro	Solicitar livro	Pré-condição
UC_OMI_03	Faltou escrever o fluxo de exceção livro já cadastrado	Cadastrar livro	Fluxo de exceção
UC_INC_03	O fluxo de exceção 1 deveria retornar para o passo 2 do fluxo principal, já que já atingiu o limite de solicitações	Solicitar livro	Fluxo de exceção
UC_FIN_03	No passo 3 faltou descrever o ator que faz a ação	Cadastrar remédio	Fluxo de exceção

O maior número de defeitos encontrados nas especificações de *user stories* foram de Fato Incorreto (9), seguido por Ambiguidade (6), Omissão (5), Inconsistência (2) e Informação estranha (2). Nas especificações de caso de uso os defeitos mais encontrados foram Omissão (9), Ambiguidade (9) seguido por Fato incorreto (2) e Inconsistência (1). Não foram encontrados defeitos de Informação estranha. A Fig. 2 mostra o número de defeitos encontrados nas especificações de *user stories* e casos de uso distribuídas por grau de severidade. A especificação de *user story* teve maior número de defeitos do tipo Médio (17), seguido de Grave (5) e Baixo (2). Vale ressaltar que o tipo Grave na *user story* foi a omissão de cenários nas especificações. Na especificação do grupo 2 foram encontrados 8 defeitos, onde 4 eram omissões de cenários importantes para o desenvolvimento da funcionalidade. A especificação de caso de uso teve maior número de defeitos do tipo Médio (10), seguido de Grave (9) e Baixo (2). Os defeitos do tipo grave em caso de uso estão relacionados a omissão de fluxo de exceção e regras de negócio.

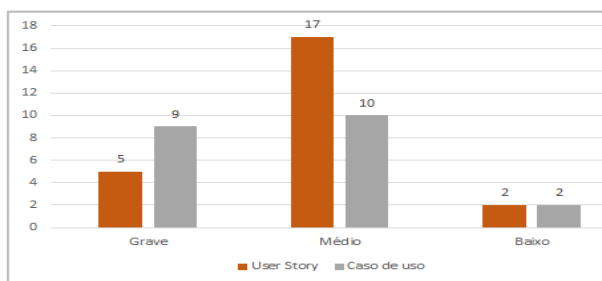


Figura 2: Número de defeitos por severidade

4.1.2 Análise dos mockups. A avaliação dos mockups em relação à especificação resultou em 25 defeitos, dos quais, 11

foram identificados nos mockups construídos a partir de especificação de *user story* e 14 identificados nos mockups construídos a partir de especificação de caso de uso. A Tabela 6 mostra o número dos diferentes tipos de defeitos identificados na inspeção dos mockups.

Tabela 6: Defeitos encontrados nos mockups

Especificação	Onde tá?	Não era assim!	Falta de dependência!	O que é isso?
<i>User story</i>	2	4	1	4
Caso de uso	4	4	4	2

Nos mockups criados a partir de especificação de *user story*, o maior número de defeitos encontrados foram do tipo “Não era assim!” (4) e “O que é isso?” (4), seguido de “Onde tá?” (2) e “Falta de dependência” (1). Nos mockups criados a partir de especificação de caso de uso, o maior número de defeitos foram do tipo “Onde tá?”, “Não era assim!” e “Falta de dependência” com 4 defeitos cada, seguido de “O que é isso?”, com 2.

Além disso, foi realizada uma avaliação da completude dos mockups criados pelos grupos em relação ao que foi especificado. Para avaliar a completude dos mockups, foi verificado se cada informação definida na especificação estava representada de forma completa no mockup. Cada item identificado como incompleto impactou na completude do mesmo, os resultados são apresentados na Fig. 3.

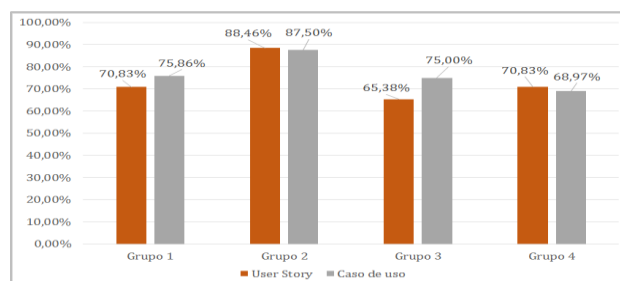


Figura 3: Corretude dos mockups criados

Percebe-se que não houve diferença significativa quanto à completude dos mockups. A completude dos mockups criados baseados em *user stories* pelos grupos 2 e 4 foi um pouco maior do que utilizando caso de uso. Já os grupos 1 e 3 tiveram um desempenho melhor utilizando o caso de uso como base para a construção do mockup.

4.1.3 Análise de propagação de defeitos. Quanto a questão de propagação de defeitos, ou seja, a quantidade de defeitos inseridos nos mockups devido a defeitos inseridos previamente nas especificações, destaca-se que enquanto os mockups criados a partir de *user stories* apresentaram 36% de defeitos nesta situação, os mockups criados a partir de casos de uso apresentaram 0% de defeitos nesta situação. Nesta seção, serão apresentadas algumas respostas para esta questão.

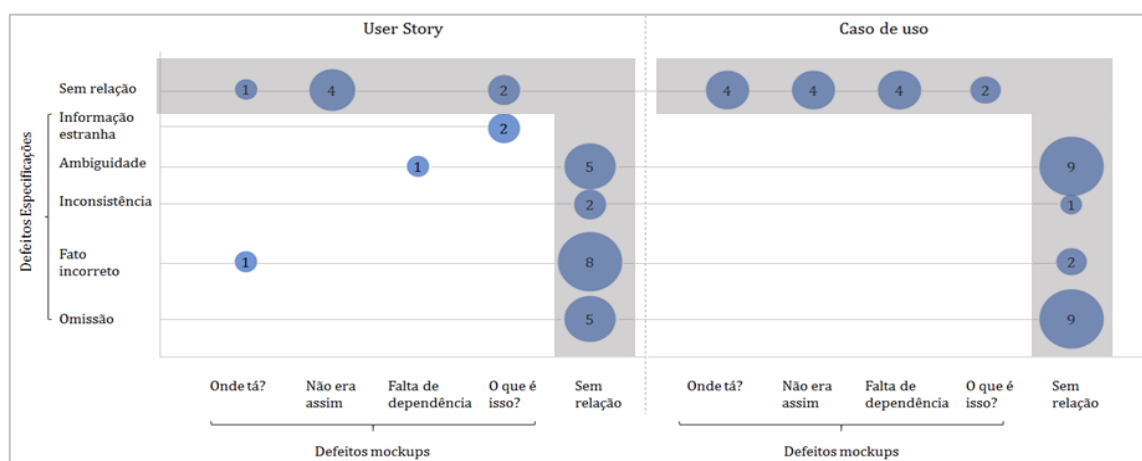


Figura 4: Defeitos especificações x defeitos mockups

Defeitos inseridos na especificação de user stories: com o intuito de prover uma visão geral sobre defeitos inseridos nos mockups e defeitos inseridos na especificação, a Fig. 4 apresenta um gráfico com a frequência de cada defeito, bem como, a relação entre eles. A área do gráfico com fundo cinza destaca os defeitos existentes nos mockups e especificações que não possuem relação entre si.

Quanto aos defeitos que possuem relação com propagação de defeitos, destaca-se que, dos 11 defeitos encontrados nos mockups, 4 (36%) foram gerados a partir de defeitos nas especificações de *user stories*, os quais estão detalhados na Fig. 5.

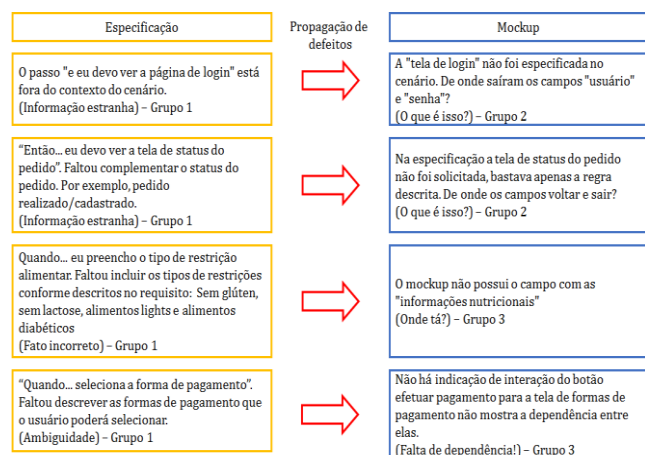


Figura 5: Defeitos propagados para mockups

Quanto a propagação de defeitos dos mockups e especificações, a área do gráfico com fundo cinza da Fig. 4, mostra que enquanto os mockups possuem 7 (74%) defeitos nesta situação – “Onde está?” (1), “Não era assim” (4), “O que é isso?” (2), as especificações possuem 20 (83%) defeitos nesta situação – Omissão (5), Fato incorreto (8), Inconsistência (2) e Ambiguidade (5). Estes resultados indicam que a maioria dos defeitos não possuem relação de propagação.

Devido à especificação do cenário 2 ter sido simplista, possuindo um alto grau de omissões de *user stories*, os mockups apresentaram poucos defeitos de construção, pois a simplicidade da especificação deixava pouca margem para defeitos. Neste caso, os grupos não inseriram nenhum defeito ao construir os mockups.

Defeitos inseridos na especificação de Caso de Uso: com o intuito de prover uma visão geral sobre defeitos inseridos nos mockups e defeitos inseridos na especificação, a Fig. 4 apresenta um gráfico com a frequência de cada defeito. O gráfico mostra, também, que não houve relação de propagação de erro ao utilizar especificação de caso de uso.

Nenhum defeito encontrado nos mockups foram gerados pelos defeitos na especificação de caso de uso. Os defeitos encontrados nos mockups criados pelos grupos foram relacionados a: não inserções de informações que estavam descritas, alteração de mensagem no mockup, inserção de botões não especificados; não indicação da navegabilidade entre as telas e apresentação dos campos de forma incorreta.

Foi observado que os defeitos dos mockups não estavam relacionados aos defeitos inseridos na especificação dos cenários, assim como mostra a Fig. 4. Os defeitos com maior frequência nas especificações foram: Omissão (9) e Ambiguidade (9).

Não houve propagação do defeito de Omissão inserido na especificação, pois como não havia informação especificada, os mockups construídos não consideraram tais informações, portanto não houve como errar. Assim, os grupos não inseriram nenhum defeito ao construir o mockup.

Quanto aos defeitos nas especificações relacionados a Ambiguidade, destaca-se que uma das técnicas definidas na Seção 3.2 Execução do estudo – utilização de bilhete de comunicação para tirar dúvidas – quando utilizada evitou a inserção dos defeitos do tipo “O que é isso?” (3) e “Não era assim” (1) nos mockups. Esta situação indica a importância da equipe esclarecer defeitos do tipo “Ambiguidade” com o emissor da especificação e não tomar decisões por conta própria. Em projetos reais pode-se substituir os bilhetes por e-mails, reuniões ou videoconferências.

No contexto deste artigo, o termo emissor foi utilizado para representar o participante que especifica os requisitos dos clientes no formato de caso de uso ou *user story*. E para o participante que recebe a especificação para desenvolver sua atividade foi utilizado o termo receptor [21].

4.2 Resultados Qualitativos

Nesta seção, são apresentados os resultados de uma análise qualitativa das respostas dos participantes (P) aos questionários aplicados ao longo da execução do estudo, com o objetivo de avaliar a utilização das especificações de caso de uso e *user story* para a comunicação de requisitos. Os métodos qualitativos segundo Seaman [26] apoiam uma melhor compreensão das questões que necessitam de uma análise mais específica e detalhada, permitindo ao pesquisador considerar o comportamento humano e entender completamente o objeto estudado. A análise qualitativa realizada neste trabalho baseia-se em procedimentos de codificação de *Grounded Theory* [30], mais detalhes no relatório técnico [23].

Enquanto eram analisados os dados contidos no questionário, foram criados códigos associados com fragmentos de texto. Outro pesquisador analisou os códigos relacionados com as citações em cada transcrição do questionário. Este pesquisador verificou os códigos e categorias para validar o processo de codificação e, portanto, mitigar o viés eventualmente causado pela participação de um único pesquisador no processo de codificação.

4.2.1 Análise dos resultados sobre especificação. Foram identificadas as dificuldades encontradas pelos participantes (P) na utilização dos modelos de especificação:

- Dificuldade com o campo 'quando' da *user story* (ver citação de P04 e P05 abaixo).

"(...) Um pouco de dificuldade para especificar as pré-condições que algumas vezes têm semelhança com os passos do "quando" – P04

"(...) esqueço as vezes a função do campo "quando" – P05

- Dificuldade com os fluxos do caso de uso (ver citação de P10 e P16 abaixo).

"(...) quando apareceu uma condição (se seria um fluxo alternativo ou um fluxo de exceção)" – P10

"(...) nos momentos de especificar os fluxos alternativos (como verificar o passo em que este será chamado)" – P16

Os participantes que apresentaram dificuldades na utilização da estrutura de especificação das *user stories* e casos de uso indicaram que pode haver um problema no modelo de especificação ou a inexperiência nos modelos pode atrapalhar na especificação dos requisitos.

Quanto à comunicação de requisitos através de *user stories*, um participante afirmou que *user story* é capaz de comunicar requisitos para todos os integrantes da equipe de desenvolvimento (P01). Os participantes P03 e P04 afirmaram que depende do tamanho do projeto e equipe, como mostram as citações abaixo:

"(...) cada cenário mostra as regras que devem ser seguidas/inseridas no sistema" – P01

"(...) user story é super eficaz para projetos pequenos de

desenvolvimento de software" – P03

"(...) depende muito do tamanho da equipe e da forma de trabalho" – P04

No entanto, os participantes P02 e P05 afirmaram que *user story* não é suficiente para comunicar requisitos, pelas seguintes razões:

"Não, pois existem vários outros passos que não ficam totalmente claros para seguir" – P02

"Não é tão bom para representar os fluxos de exceção" – P05

Quanto a comunicação de requisitos utilizando a especificação de caso de uso somente um participante (P16) discordou que casos de uso são capazes de comunicar requisitos para todos os integrantes da equipe de desenvolvimento, como:

"a especificação não detalha exatamente a consequência das ações tomadas pelo usuário, por exemplo, além de agrupar muitas informações em um único local ou que pode gerar confusão ao seguir os fluxos" – P16

Ao verificar se os participantes que utilizaram *user story* para especificar utilizariam outro artefato complementar, foram listados os seguintes artefatos que poderiam tornar a especificação mais completa: Protótipo (P01), Detalhes do usuário (P02), Caso de uso (P03, P05 e P06) e Personas (P04, P07 e P08). Os que utilizaram a especificação de caso de uso apontaram os seguintes artefatos que poderiam tornar a especificação mais completa: Diagrama de caso de uso (P11), Protótipos (P15) e *User stories* (P09, P12 e P16).

4.2.2 Análise dos resultados sobre criação de mockups. Quanto à utilização das especificações de caso de uso e *user story* para a criação dos mockups foram encontradas três categorias de problemas de comunicação: problema de especificação por parte do emissor, problema no modelo de especificação e problema de entendimento do modelo de especificação. A Tabela 7 mostra algumas citações referentes a estes problemas. Todas as citações relacionadas são mostradas em [23].

Tabela 7: Exemplos de códigos sobre problemas

Categorias	Citações
Problema de especificação por parte do emissor	<i>"(...) alguns fluxos pareciam que levavam para duas telas ao mesmo tempo"</i> – P05 - UC <i>"Na construção do mockup percebemos que algumas informações estavam faltando"</i> – P10 - US
Problema no modelo de especificação	<i>"(...) excesso de fluxos, algum elemento pode passar batido, pois as informações ficam espalhadas"</i> – P16 - UC <i>"a especificação é falha em algumas informações que são necessárias em cenários adjacentes"</i> – P09 - US
Problema de entendimento do modelo de especificação	<i>"(...) como tinha fluxo alternativos, foi necessário buscar informações fora do fluxo principal e isso dificulta a extração"</i> - P11 - UC <i>"tive dificuldade na parte de "dado que" pois fiquei em dúvida se os dados eram pré-condições ou se eram para ser inseridos"</i> - P05 - US

4.2.3 Análise dos resultados gerais. Os resultados do

questionário final, verificou-se na percepção dos participantes qual das especificações, caso de uso ou *user story*, detalhava mais as informações para a construção dos mockups. Os argumentos relacionados às preferências por *user story* foram:

“(...) user story mostra melhor as interações das ações” - P02

“(...) user story foi mais detalhista, pela possibilidade de criar cenários separados” - P04

Os argumentos relacionados às preferências por caso de uso foram:

“(...) a especificação com mais detalhes de informação para construção de mockup é o caso de uso, porque as informações de regra de negócio, por exemplo, deixam mais detalhada” - P10

“(...) casos de uso detalham melhor todos os fluxos, fornecendo maiores detalhes das ações” - P16

Vale ressaltar que dos 8 participantes que especificaram com *user story* somente 3 acham que caso de uso detalha mais as informações para a construção dos mockups e dos 8 participantes que especificaram com caso de uso, somente um acredita que *user story* detalha mais para construção do mockup.

5 DISCUSSÃO

Durante a análise da totalidade de defeitos encontrados, a *user story* totalizou 24 defeitos contra 21 de casos de uso. Apesar da especificação de caso de uso ter apresentado 9 omissões contra 5 da *user story*, os problemas gerados com as omissões da *user story* foram mais impactantes, pois tratava-se de informações essenciais para a construção dos mockups.

Comparando o número de defeitos encontrados nos mockups identificados na inspeção, foi observado o total de 11 defeitos nos mockups criados utilizando *user story* e 14 defeitos nos mockups criados utilizando UC. Os mockups criados com caso de uso tiveram mais defeitos do tipo “Onde tá?”, “Não era assim!” e “Falta de dependência!”. Esses defeitos podem ter sido ocasionados por problema de entendimento do modelo de especificação.

Como mostrado na Tabela 7, os problemas de comunicação de requisitos encontrados foram: 1) Problema de especificação por parte do emissor, 2) Problema no modelo de especificação e 3) Problema de entendimento do modelo de especificação. Estes problemas podem ter sido ocasionados pela falta de experiência dos participantes nestes tipos de especificação. Sugerindo assim que o fator humano não deve ser negligenciado na dinâmica da comunicação de requisitos dentro de equipes de desenvolvimento.

Neste estudo, existiram algumas ameaças que podem afetar a validade dos resultados e que foram mitigadas quando possível. As principais ameaças foram: (1) Efeitos de treinamento: os participantes receberam treinamento equivalente nos dois tipos de especificação, incluindo atividades teóricas e exercícios práticos em sala de aula; (2) Uso de cenários: foi minimizada utilizando cenários escritos em linguagem natural, onde os requisitos deste cenário estavam explícitos, de forma similar aos exercícios realizados durante o treinamento; (3) Representatividade dos participantes: embora os sujeitos deste estudo fossem estudantes, Höst [14] afirma que estudantes podem representar uma população de profissionais da indústria;

(4) Tamanho e homogeneidade da amostra: devido ao número limitado de participantes, também uma vez que todos os participantes eram estudantes da mesma instituição, os resultados deste estudo não podem ser considerados conclusivos.

6 CONCLUSÃO

Este artigo apresentou um estudo experimental exploratório, que comparou a dinâmica da comunicação de requisitos e seus resultados utilizando especificação de caso de uso e *user stories* como base para a criação de mockups. A questão de pesquisa investigada foi avaliar se a comunicação de requisitos para a construção de mockups é afetada de forma diferente ao se usar as especificações de casos de uso ou *user stories*.

De acordo com a análise realizada sobre os resultados alcançados, foi identificado que a especificação de caso de uso gerou mais defeitos na parte da construção dos mockups e menos defeitos na parte de especificação. Entretanto, destaca-se que a quantidade e o impacto desses defeitos no resultado final não são suficientes para determinar qual das duas especificações é melhor ou pior para a comunicação de requisitos entre equipes de desenvolvimento de software. Os resultados mostraram que não há diferença significativa que sustente escolher por uma forma de especificação em detrimento da outra. Então, sob esta ótica, as equipes de desenvolvimento de software que estiverem com dúvida em qual das formas de especificação adotar, podem optar tanto pela utilização de *user story* quanto caso de uso.

Percebe-se, também, que vários defeitos encontrados nos mockups não foram causados por defeitos na especificação, mas originaram-se por fatores relacionados à proatividade do receptor sem considerar o que estava especificado e sem considerar a necessidade de validação por parte do emissor. Este tipo de proatividade deve ser desencorajado, pois pode gerar mais problemas do que benefícios.

No entanto, foi observado que a partir da análise geral dos resultados, indícios que o fator humano foi uma das causas de geração e propagação dos defeitos, além da parte técnica. Para minimizar esses tipos de problemas, deve-se identificar quais os defeitos possuem natureza essencialmente técnicas e quais defeitos são advindos de fatores humanos. Os defeitos de natureza técnicas estão relacionados com o artefato utilizado para especificação dos requisitos e poderão ser evitados ou eliminados através de treinamentos, definições de padrões (nomenclaturas, escritas, templates) e inspeção nas especificações. Os defeitos de natureza humana estão relacionados com o profissional que utilizou o artefato em suas atividades, podem estar relacionados à falta de conhecimento do domínio do problema, falta de experiência, falta de compromisso e deve-se procurar meios de minimizar esses tipos de interferência.

Neste estudo, existiram algumas limitações que devem ser consideradas. As principais limitações foram: (1) utilização de cenários com níveis de complexidade baixos - foi adotado esse nível de complexidade pois o estudo foi realizado em ambiente acadêmico e o tempo para execução das atividades foi limitado; (2) aspectos avaliados - tratou-se a representação das funcionalidades de interação e não o aspecto de interferência

entre requisitos; (3) avaliação da comunicação - a comunicação entre diferentes stakeholders não foi abordada, pois o foco foi apenas na comunicação dentro da equipe de desenvolvimento. Essas limitações são pontos a serem melhorados em estudos futuros.

Como trabalhos futuros, pretende-se reaplicar este estudo com novos alunos e na indústria para identificar outras causas de problemas de comunicação de requisitos. E, com o objetivo de verificar como ocorre a comunicação de requisitos na fase de implementação e testes, pretende-se entender o experimento para outras fases do ciclo de vida do desenvolvimento de software. Pretende-se, também, investigar quais fatores humanos são capazes de influenciar a propagação de defeitos na comunicação de requisitos em equipes de desenvolvimento de software e como podemos minimizar esse tipo de problema. Além disso, pretende-se realizar um levantamento sobre as principais técnicas que utilizam linguagem do contexto para comparar o resultado com as técnicas deste trabalho.

AGRADECIMENTOS

Os autores agradecem o apoio financeiro do CNPq processo 423149/2016-4, CAPES processo 175956/2013, da FAPEAM, processo 062.00578/2014, da FAPERJ (projetos E-26/010.000883/2016, E-211.174/2016), UNIRIO (PQ-UNIRIO 01/2016 e 01/2017) e também aos participantes do estudo.

REFERÊNCIAS

- [1] A. Al-Rawasa and S. Easterbrook. 1996. Communication problems in requirements engineering: A field study. In *Proc. of Conf. on Professional on Awareness in Software Engineering*, London, 47–60.
- [2] B. Anda, H. Dreiem, D. Sjøberg and M. Jørgensen. 2001. Estimating software development effort based on use cases – experiences from industry. In *UML 2001 - The Unified Modeling Language. Modeling Languages, Concepts, and Tools*. Springer, Berlin Heidelberg, v. 2185, 487–502.
- [3] B. Anda, K. Hansen and G. Sand. 2009. An investigation of use case quality in a large safety-critical software development project. In *Information and Software Technology*, v. 51, n. 12, 1699–1711. DOI: <https://doi.org/10.1016/j.infsof.2009.04.005>.
- [4] B. Anda and D. Sjøberg. 2002. Towards an inspection technique for use case models. In *Proc. of the 14th Intl. Conf. on Software Engineering and Knowledge Engineering*, ACM, NY, USA, 127–134. DOI: <http://dx.doi.org/10.1145/568760.568785>.
- [5] A. Belgamo and Luiz E. G. Martins. 2000. Estudo comparativo sobre as técnicas de elicitação de requisitos do software. In *XX Congresso Brasileiro da Sociedade Brasileira de Computação (SBC)*.
- [6] E. Bezerra. 2007. Princípios de Análise e Projeto de Sistemas com UML, Campus, 2ª edição.
- [7] A. Ceverino e Fernando P. Nascimento. 2016. Utilização da técnica de desenvolvimento orientado por comportamento (BDD) no levantamento de requisitos. *Revista Interdisciplinar Científica Aplicada*, v.10, n.3, 40-51, TRIII 2016. ISSN 1980-7031.
- [8] D. Chelmsky, D. Astels, B. Helmkamp, D. North, Z. Dennis and A. Hellesoy. (2010). The RSpec Book: Behaviour Driven Development with Rspec, Cucumber, and Friends, Pragmatic Bookshelf.
- [9] A. Cockburn. 2001. *Writing Effective Use Cases*, Vol. 1, Addison-Wesley, Boston.
- [10] E. Evans. 2003. *Domain-Driven Design: Tackling Complexity In the Heart of Software*. Boston, MA, USA: Addison-Wesley.
- [11] Daniel M. Fernández, S. Wagner, M. Kalinowski, M. Felderer, P. Mafra, A. Vetrò, T. Conte, et al.. 2016. Naming the pain in requirements engineering. *Empirical Software Engineering*, 1-41. DOI: <http://dx.doi.org/10.1007/s10664-016-9451-7>.
- [12] K. Gohil, N. Alapati and S. Joglekar. 2011. Towards behavior driven operations (bdops). In: *Intl. Conf. on Advances in Recent Technologies in Communication and Computing*, 262–264.
- [13] B. Hoisl, S. Sobernig and M. Strembeck. 2014. Comparing Three Notations for Defining Scenario-based Model Tests: A Controlled Experiment In *9th Intl. Conf. on the Quality of Information and Communications Technology*

(QUATIC 2014), 180-189. DOI: <https://doi.org/10.1109/QUATIC.2014.62>.

- [14] M. Höst, B. Regnell and C. Wohlin. 2000. Using Students as Subjects – A Comparative Study of Students and Professionals in Lead-Time Impact Assessment. In: *Empirical Software Engineering*, v. 5, n. 3, 201–214.
- [15] I. Ibrwesh, Sin-Ban Ho, I. Chai and Chui-Hong Tan. 2017. A Controlled Experiment on Comparison of Data Perspectives for Software Requirements Documentation. *Arabian Journal for Science and Engineering*, 1-15.
- [16] I. Jacobson. 1987. Object-oriented development in an industrial environment. In *Conf. on Object-Oriented Programming, Systems, Languages & Applications*, 183–191.
- [17] S. Lauesen and Mohammad A. Kuhail. 2012. Task descriptions versus use cases. *Requirements Engineering*, v. 17, n. 1, 3-18. DOI: <https://doi.org/10.1007/s00766-011-0140-1>
- [18] Esteban R. Luna, José I. Panach, J. Grigera, et al. 2010. Incorporating usability requirements in a test/model-driven web engineering approach. In: *Journal of Web Engineering*, v. 9, n. 2, 132–156.
- [19] M. Cohn. 2004. *User stories applied: For agile software development*. Addison-Wesley Professional.
- [20] Rafael M. Mello, Eldānae N. Teixeira, M. Schots, Cláudia M. L. Werner and Guilherme H. Travassos. 2014. Verification of Software Product Line Artefacts: A Checklist to Support Feature Model Inspections, *Journal of Univ. Comp. Science*, v. 20, 720-745.
- [21] V. Mikulovic and M. Heiss. 2006. How do I know what I have to do?: the role of the inquiry culture in requirements communication for distributed software development projects. In *Proc. of the 28th Intl. Conf. on Software engineering*, 921–925. DOI: <https://doi.org/10.1145/1134285.1134453>.
- [22] P. Mohagheghi, B. Anda, R. Conradi. 2005. Effort estimation of use cases for incremental large-scale software development. In *27th Intl. Conf. on Software Engineering*, 303–311. DOI: <https://doi.org/10.1109/ICSE.2005.1553573>
- [23] Ana C. Oran, E. Nascimento, G. Santos e T. Conte. 2017. Relatório técnico: Uma Análise sobre Comunicação de Requisitos Utilizando Especificação de Caso de Uso e User story. TR-USES-2017-0010. Disponível em: <http://uses.icomp.ufam.edu.br/relatorios-tecnicos/>.
- [24] Keith T. Phalp, J. Vincent and K. Cox. 2007. Assessing the quality of use case descriptions. In *Software Quality Journal*, v. 15, n. 1, 69–97. DOI: <http://dx.doi.org/10.1007/s11219-006-9006-z>.
- [25] Raquel O. Prates, Clarisse S. de Souza and Simone D. Barbosa. 2000. Methods and tools: a method for evaluating the communicability of user interfaces interactions, v. 7, n. 1, 31-38.
- [26] Carolyn B. Seaman. 1999. Qualitative methods in empirical studies of software engineering. *IEEE Transactions on software engineering*, v. 25, n. 4, 557-572.
- [27] Thiago R. Silva. 2016. Definition of a behavior-driven model for requirements specification and testing of interactive systems. In *Requirements Engineering Conf.*, IEEE 24th Intl., 444-449. DOI: <https://doi.org/10.1109/RE.2016.12>
- [28] J. Smart. 2014. *BDD in Action: Behavior-Driven Development for the Whole Software Lifecycle*. New York, USA: Manning Publications.
- [29] K. Stapel, E. Knauss and K. Schneider. 2009. Using flow to improve communication of requirements in globally distributed software projects. In *Requirements: Communication, Understanding and Softskills*. Collaboration and Intercultural Issues, 5-14. IEEE. DOI: <http://dx.doi.org/10.1109/CIRCUS.2009.6>
- [30] A. Strauss and J. Corbin. 2014. *Basics of Qualitative Research: Techniques and Procedures for Developing Grounded Theory*, in Thousand Oaks, CA, SAGE publications.
- [31] S. Tiwari and A. Gupta. 2015. A systematic literature review of use case specifications research. In *Information and Software Technology*, v. 67, 128–158. DOI: <https://doi.org/10.1016/j.infsof.2015.06.004>.
- [32] Guilherme H. Travassos, F. Shull, M. Fredericks and V. Basili. 1999. Detecting defects in object-oriented designs: using reading techniques to increase software quality. In *Proc. of XIV ACM SIGPLAN conf. on Object-oriented programming, systems, languages, and applications*, v. 34, n. 10, 47-56. DOI: <https://doi.org/10.1145/320384.320389>.
- [33] Yu-Cheng Tu, E. Tempero and C. Thomborson. 2016. An experiment on the impact of transparency on the effectiveness of requirements documents. *Empirical Software Engineering*, v. 21, n. 3, 1035-1066. DOI: <http://dx.doi.org/10.1007/s10664-015-9374-8>.
- [34] X. Wang, L. Zhao, Y. Wang and J. Sun. 2014. The Role of Requirements Engineering Practices in Agile Development: An Empirical Study. In *Proc. of the Asia Pacific Requirements Engineering Symposium*, ser. CCIS. Springer, v. 432, 195–209.
- [35] A. Zeaaraoui, Z. Bougroun, M. G. Belkasmí and T. Bouchentouf. 2013. *User stories template for object-oriented applications*. In *Innovative Computing Technology (INTECH)*, 407–410.